A genetic algorithm for learning the parameters of an SRMP preference model

Bastien Pasdeloup[1], Arwa Khannoussi[2], Alexandru-Liviu Olteanu[3], Patrick Meyer[1]

[1] IMT Atlantique, Lab-STICC, UMR CNRS 6285
[2] IMT Atlantique, LS2N, UMR CNRS 6004
[3] Lab-STICC, Université Bretagne Sud

DA2PL 2022, 17-18 November 2022, Université de Technologie de Compiègne - France

Outline of the presentation

- SRMP ?
- A genetic algorithm to infer the preference parameters
- Hyperparameter tuning
- A “tuned” multi-operator genetic algorithm
- Experiments

SRMP: Simple Ranking Method using Reference Profiles

SRMP

- SRMP is a Multi-Criteria Decision Aiding (**MCDA**) technique based on the **outranking paradigm**
- **Ranks** alternatives according to the preferences of a decision maker (**DM**)
- *Little refresher*: *a* **outranks** *b* ($a \succeq b$, *a is at least as good as b*) iff *a* is at least as good as *b* on a weighted majority of criteria
- SRMP prevents cycles / intransitivities that could result from such pairwise comparisons of all the alternatives

General idea of SRMP

Compare alternatives indirectly through **reference points** which can be seen as *norms*

- $\mathcal{A}$ is a set of n alternatives evaluated on a set of m criteria $M = \{1, \ldots, m\}$
- **Preference parameters** in SRMP:
  - k reference profiles: $p^1, \ldots, p^h, \ldots, p^k$
    $p^h \equiv (p^h_1, \ldots, p^h_m)$
  - a lexicographic order σ on the reference profiles
  - criteria weights $w_1, \ldots, w_m$, with $w_j \geqslant 0$ and $\sum_{j \in M} w_j = 1$

- $f_w(a, p^h)$ represents the **strength** of the criteria coalition supporting the statement $a \succeq p^h$:

$$f_w(a, p^h) := \sum_{j \in C(a,p^h)} w_j$$

where $C(a, p^h) = \{j \in M : a \succsim_j p^h\}$

Figure: $f_w(a, p^1) = w_1 + w_3$

SRMP through an illustrative example

- Job offers (x,y and z) have to be **ranked** according to a **candidate's preferences**
- x, y and z are evaluated on **three criteria**:
  - *salary* (S) (in k€, the higher, the better)
  - *location* (L) (in km, the lower, the better)
  - *job appeal* (J) (good $\succsim_J$ fair $\succsim_J$ poor $\succsim_J$ very poor)
- Data & preference parameters :

	S	L	J
x	41	2	good
y	46	4	poor
z	43	5.5	fair

Performance table

	S	L	J
p^1	42	5	poor
p^2	45	3	good
w	1/3	1/3	1/3

Reference profiles and weights

$\sigma = (1, 2)$

SRMP through an illustrative example

Comparisons w.r.t. p^1 :

$$\left.\begin{array}{l} f_w(x,p^1) = \sum_{j \in C(x,p^1)} w_j \\ \qquad\quad = 0 + 1/3 + 1/3 = 2/3 \\ f_w(y,p^1) = 1/3 + 1/3 + 1/3 = 1 \\ f_w(z,p^1) = 1/3 + 0 + 1/3 = 2/3 \end{array}\right\} \Rightarrow \begin{array}{l} y \succ_{p^1} x \\ y \succ_{p^1} z \\ x \sim_{p^1} z \end{array}$$

Comparisons w.r.t. p^2 :

$$\left.\begin{array}{l} f_w(x,p^2) = 0 + 1/3 + 1/3 = 2/3 \\ f_w(z,p^2) = 0 + 0 + 0 = 0 \end{array}\right\} \Rightarrow x \succ_{p^2} z$$

Final ranking: $y \succ x \succ z$

SRMP: preference elicitation

- Determination of the **values of the parameters** of the SRMP model compatible with the DM's preferences

- **Indirect elicitation**
  - Determine these values from holistic judgments of the DM on pairs of alternatives (aPb / aIb)
  - Mixed Integer Program, Boolean satisfiability problem, Meta-heuristic
  - In a **batch**[1][2][3] or **incremental** [4] setting

[1] A.L. Olteanu, K. Belahcene, V. Mousseau, W. Ouerdane, A. Rolland, and J. Zheng, Preference elicitation for a ranking method based on multiple reference profiles, 4OR, 20(1), 63–84, 2022

[2] K. Belahcene, V. Mousseau, W. Ouerdane, M. Pirlot, O. Sobrie, Ranking with Multiple Reference Points: Efficient SAT-based learning procedures. Computers and Operations Research, 2023

[3] J. Liu, W. Ouerdane, and V. Mousseau, A metaheuristic approach for preference learning in multicriteria ranking based on reference points, in Proceedings of the 2nd DA2PL workshop, pp. 76–86, 2014

[4] A. Khannoussi, A.L. Olteanu, C. Labreuche, P. Meyer, Simple Ranking Method using Reference Profiles : Incremental elicitation of the preference parameters, 4OR, 2021

SRMP: preference elicitation

SRMP: preference elicitation

- **However**:
  - **Computational effort** not compatible with a live elicitation process
    (i.e. **quite long "waiting time"** *between two queries)*

- **Our proposal**:
  - Return temporarily to the **batch** setting
  - To **design** a **genetic algorithm** (GA) to infer the preference parameters of SRMP
  - Perspective: integrate the GA in an **incremental** setting later

A genetic algorithm to infer the preference parameters

The genetic algorithm

- A **solution** / **chromosome** consists of the performances of the reference profiles, criteria weights and a lexicographic order of the reference profiles.

The genetic algorithm

- Totally approximate but comprehensive flowchart

The genetic algorithm

- The **fitness function**: percentage of input statements (holistic judgements of the DM) correctly restored by the solution (**train accuracy**)

The genetic algorithm

- The **stopping criterion**: no evolution of the best s solutions during 50 iterations

- Hyperparameter : s

The genetic algorithm

- Many “choices”

The genetic algorithm

The genetic algorithm

- Crossover operators

Figure: Swap weights

The genetic algorithm

- Crossover operators

Figure: Swap (lexicographic) orders

The genetic algorithm

- Crossover operators

Figure: Swap profiles

The genetic algorithm

- Crossover operators

Figure: Mix “criteria”

- Hyperparameter : number of criteria in the “mix” ((uniform) random choice here)

The genetic algorithm

- Crossover operators

Figure: Mix “criteria” and weights

- Hyperparameter : number of criteria in the “mix” ((uniform) random choice here)

The genetic algorithm

The genetic algorithm

- Mutation operators

Figure: Random weights perturbation

- Hyperparameter : perturbation factor

The genetic algorithm

- Mutation operators

Figure: Random profiles perturbation

- Hyperparameter : perturbation factor

The genetic algorithm

- Mutation operators

Figure: Expand profiles

- Hyperparameters : expand factor, number of criteria affected (all in this study)

The genetic algorithm

- Mutation operators

Figure: Shrink profiles

- Hyperparameters : shrink factor, number of criteria affected (all in this study)

The genetic algorithm

- Mutation operators

Figure: Partially reverse (lexicographic) order

- Hyperparameter : number of criteria affected (random subset here)

The genetic algorithm

The genetic algorithm

- Elitism ratio : keep a certain amount of the **best solutions** from the **previous** population
  - Hyperparameter : amount z of best solutions to keep
- Random ratio : inject a certain amount of **randomly generated** chromosomes in the new population
  - Hyperparameter : amount y of random solutions to inject

Hyperparameter tuning

Hyperparameter tuning: the setting

- Some hyperparameters are **fixed** in advance (after a few preliminary experiments)
  - number of profiles in the sought preference model ($= k$)
  - stopping criterion: no improvement of the 10% of z best solutions during 50 iterations
  - number of parents (crossover): 2

- Remaining hyperparameters: **5** values per parameter through a **grid search**
  - Population size: $\{100, 150, 200, 250, 300\}$
  - Random ratio (y) (in the new population): $\{0.1, 0.2, 0.3, 0.4, 0.5\}$
  - Elitism ratio (z) (in the new population): $\{0.1, 0.2, 0.3, 0.4, 0.5\}$
  - Expand factor: $\{0.2, 0.4, 0.6, 0.8, 1.0\}$
  - Shrink factor: $\{0.2, 0.4, 0.6, 0.8, 1.0\}$
  - Weights perturbation factor: $\{0.1, 0.2, 0.3, 0.4, 0.5\}$
  - Profiles perturbation factor: $\{0.1, 0.2, 0.3, 0.4, 0.5\}$

  } 625×10 execs.

- On **single**-operator configurations of the GA (1 crossover, 1 mutation)

Hyperparameter tuning: the data

- **Problem size**: 11 criteria, 3 profiles $\rightarrow$ "random" DM
- **train set**: 100 pairwise comparisons on **50** alternatives (*holistic judgments of the "random" DM*)
- **test set 1**: the remaining pariwise comparisons on the 50 alternatives ($\frac{50 \cdot 49}{2} - 100$)
- **test set 2**: 300 unseen alternatives ($\frac{300 \cdot 299}{2}$ pairs)

$\times$ 10

Hyperparameter tuning: the questions

- Are there any **pairs** of crossover and mutation operators which are "better" than the others?
- For these pairs, can we determine "good" **values** for the related hyperparameters?
- What are the related **accuracies** on the train and 2 test sets?

Hyperparameter tuning: results

- **Mutation**: partially reverse (lexicographic) order, **crossover**: mix criteria

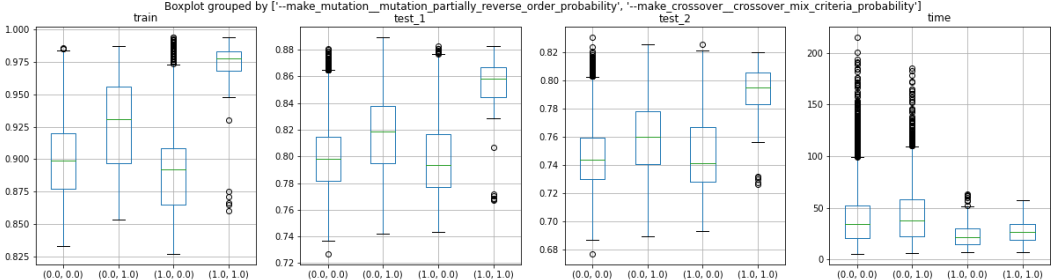


Figure: Fitness of the best solution on the 3 datasets and calculation time: all but this pair, any mutation and this crossover, any crossover and this mutation, this pair only

- Using this pair improves accuracies on the 3 datasets
- Keep only pairs with this behavior

Hyperparameter tuning: results

- 6 interesting pairs according to this first filtering + filtering through accuracy performance

mutation	crossover
random weights perturbation	mix criteria
random weights perturbation	mix criteria and weights
shrink profiles	mix criteria
shrink profiles	mix criteria and weights
partially reverse order	mix criteria
partially reverse order	mix criteria and weights

Hyperparameter tuning: results

- On these selected pairs:

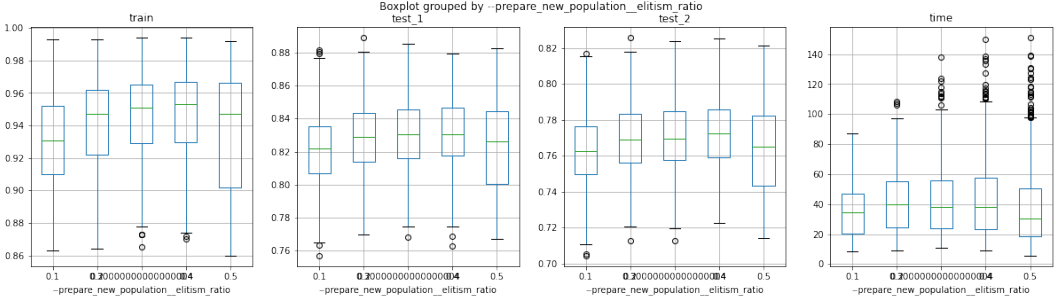


Figure: Fitness of the best solution on the 3 datasets and calculation time w.r.t. elitism ratio

- $z = 0.4$ seems to be a good candidate

Hyperparameter tuning: results

- On these selected pairs:

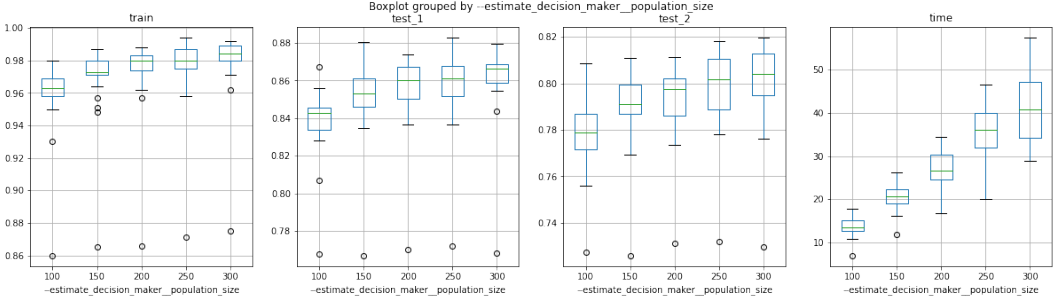


Figure: Fitness of the best solution on the 3 datasets and calculation time w.r.t. population size

- The larger the better, but calculation time increases
- Do the same with the other hyperparameters

Hyperparameter tuning: results

- Tuned values for the hyperparameters

hyperparameter	value
elitism ratio	0.4
random ratio	0.1
population size	300 (and probably above)
random weights perturbation factor	0.1
shrink factor	0.8

A multi-operator genetic algorithm to infer the preference parameters

The multi-operator genetic algorithm

- Multiple crossovers and mutations, selected randomly

Experiments

Experimental setting

- **Problem sizes**: 6, 13, 30, 100, 1000 crit., 3,6, 30, 60 prof. $\rightarrow$ "random" DM
- **train set**: 100, 300, 500, 1000, 2000, ..., 5000 pairwise comparisons on **500** alternatives
  (*holistic judgments of the "random" DM*)
- **test set 1**: the remaining pariwise comparisons on the 500 alternatives
- **test set 2**: 5000 unseen alternatives

$\times$ 10

- Mixed integer linear program (MILP), SRMP

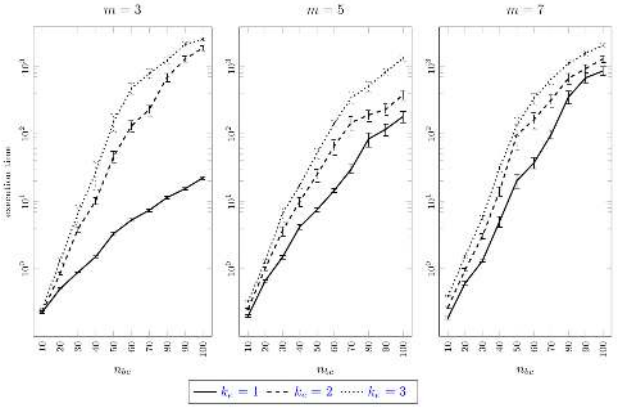


Figure: Computation times

- Boolean satisfiability formulation (SAT), RMP

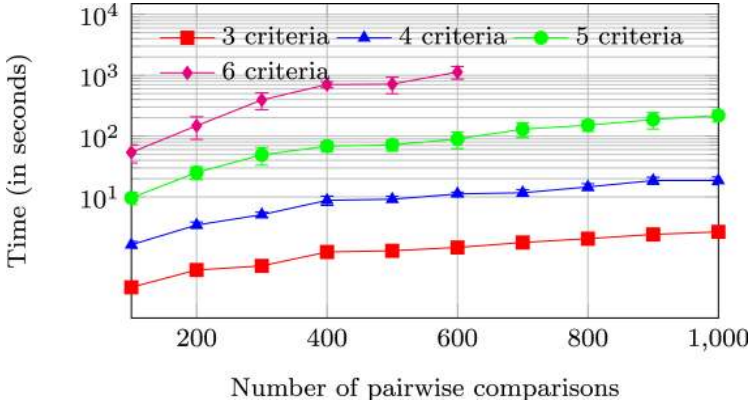
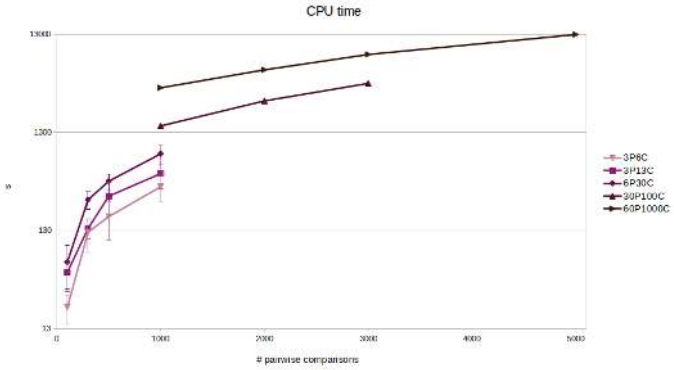


Figure: Computation times

Our proposal

- Tuned GA: CPU times for 3 problem sizes (3P6C, 3P13C, 6P30C)



- Improvement over SAT and MILP
- Handles "larger" problems (in terms of number of criteria and pairwise comparisons)

- Boolean satisfiability formulation (SAT)

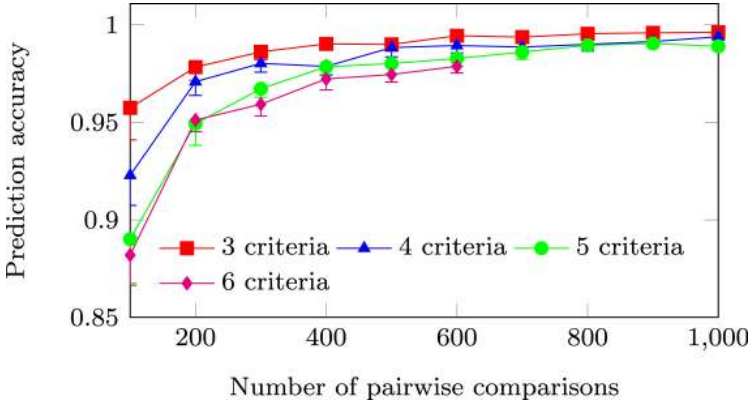
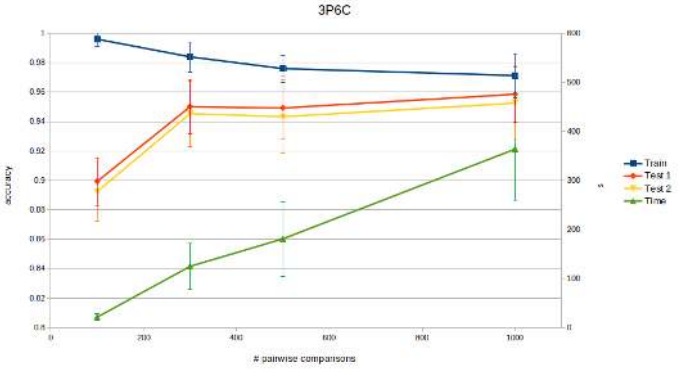


Figure: Generalisation accuracy

Our proposal

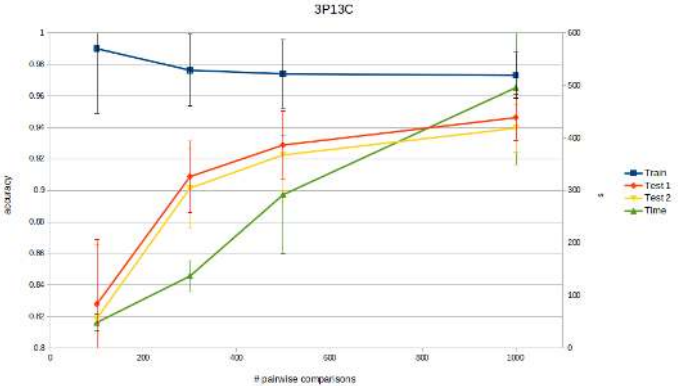
- Tuned GA: 3P6C, train and generalisation accuracies & CPU time



- Obviously lower accuracy on train set
- Similar generalization power as SAT & MILP

Our proposal

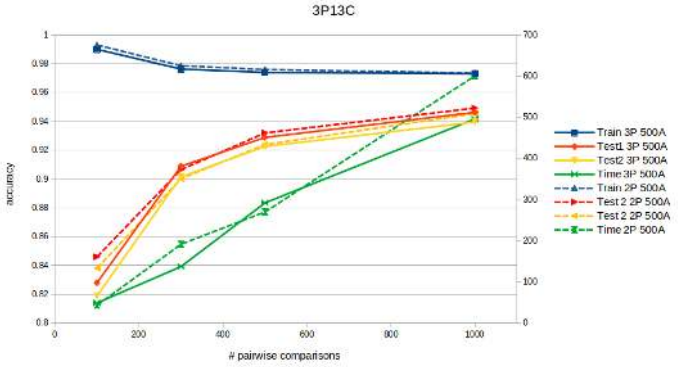
- Tuned GA: 3P13C, train and generalisation accuracies & CPU time



- Handles larger problems (in terms of number of criteria) than SAT & MILP

Our proposal

- Tuned GA: 3P13C data, 3P model vs 2P model



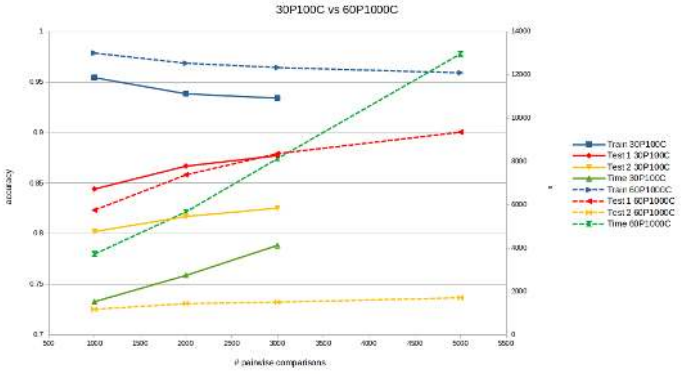
- Similar performances for generalization
- Similar computation time

} → choice of k ?

- Same conclusions for 6C, 30C

Our proposal

- Tuned GA: 30P100C vs 60P1000C, train and gen. accuracies & CPU time



- Computation times become huge (around 3.5 hours for 5000 pairwise comparisons)

Future work

Future work

- **Fine-tuning** of the hyperparameters through a greedy search around the current "best" values
- **Random search** on the continuous domains of the hyperparameters
- What about **noise** in the learning data?
- Search for "**simpler**" models with lower number of profiles & what is a good number of profiles?
- Replace random choices in the GA by **reinforcement learning**
- Integrate the GA in an **incremental** elicitation process